INDUSTRIAL ASSIGNMENT- 10 Developing Backend code using Spring Boot
STUDENT INTERNSHIP AND PLACEMENT MANAGEMENT PORTAL

1. Introduction

The backend is the part of the Student Internship and Placement Management Portal that stores the data, applies the rules and offers the REST APIs used by the web application. Industrial Assignments 1 to 9 defined the project, requirements, user stories, SQL design, ER diagram, Figma screens and the full stack setup. This assignment (Industrial Assignment 10) and the next one (Industrial Assignment 11) describe the backend that was developed with Spring Boot and tested with Postman in Industrial Assignment 12.

Industrial Assignment 10 explains the foundation of the backend: the technology, the layered architecture, the project structure, the configuration, the entity classes and database mapping, the repositories, the request and response objects (DTOs) and the security configuration. Industrial Assignment 11 explains the services, controllers, REST APIs, validation and exception handling that are built on this foundation.

2. Objectives

- Create a Spring Boot project in Java 17 with Maven and connect it to a MySQL database.
- Follow a layered structure (controller, service, repository, entity) so that every class has one clear job (NFR-09 of Industrial Assignment 2).
- Map the core data of the portal (students, companies, opportunities, applications and interviews) to JPA entities with keys, relations, unique constraints and indexes.
- Keep created and updated timestamps for every record.
- Use DTO records with Bean Validation so that invalid data never reaches the database (business rule BR-02).
- Protect the APIs with Spring Security, BCrypt-encoded passwords and role-based access for staff, students and recruiters (business rule BR-11).

3. Technology and Tools

Item	Technology	Purpose in the backend
Language	Java 17	Backend programming language
Framework	Spring Boot 3.5.7	Application startup, auto-configuration, embedded web server
Web	Spring Web (spring-boot-starter-web)	REST controllers and JSON handling
Persistence	Spring Data JPA with Hibernate	Entities, repositories and database access

Item	Technology	Purpose in the backend
Validation	Jakarta Bean Validation (starter-validation)	Rules such as @NotBlank, @Email and @Size
Security	Spring Security with BCrypt	HTTP Basic login, roles and password hashing
Database	MySQL 8 (mysql-connector-j)	Permanent storage of all records
Productivity	Lombok	@Getter, @Setter and @NoArgsConstructor on entities
Testing	Spring Boot Test and H2 (in-memory)	Integration test without a real database
Build	Maven and the Maven Wrapper (mvnw)	Dependency management, build and run
API testing	Postman	Manual and automated API tests (Industrial Assignment 12)

4. Backend Architecture

The backend follows a layered architecture. A request first passes through the security layer, which checks the login and the role. The controller receives the request, validates the data and calls a service. The service applies the business rules and uses repositories to read or write entities. The repository talks to MySQL through JPA. Results are returned to the client as DTO records, and errors are converted to JSON messages by one exception handler.

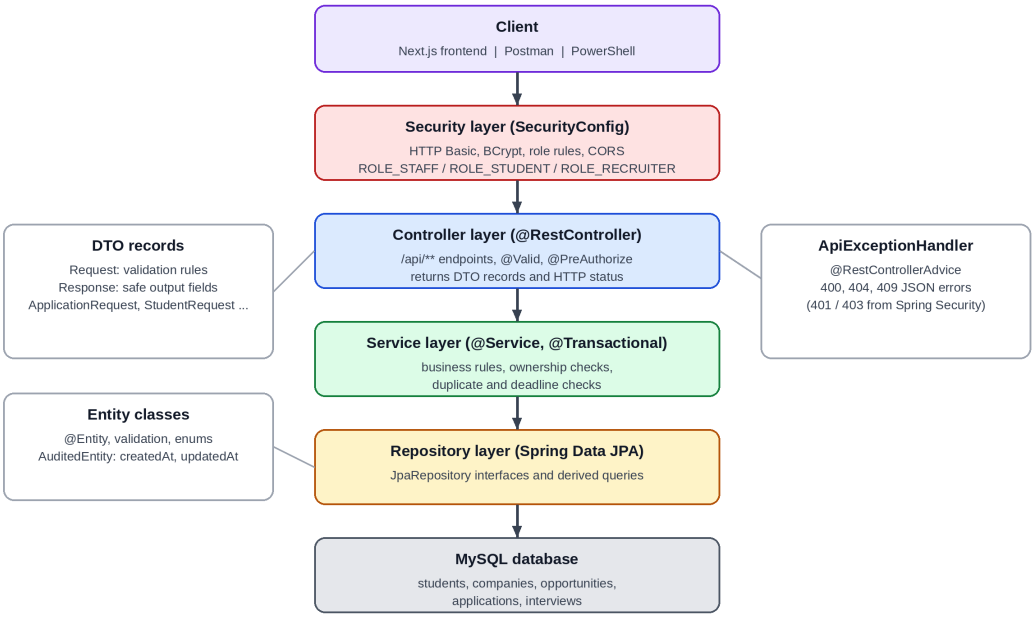


Figure 1: Layered architecture of the SIPMP backend

Layer	Responsibility
Security (config)	Decides which role may call which URL; creates the in-memory accounts; sets the CORS rule for the frontend.
Controller	Maps URLs such as /api/students to methods, checks input with @Valid and method rules with @PreAuthorize, and returns HTTP status codes.
Service	Contains the business rules: duplicate checks, deadline checks, ownership checks for students and recruiters, and conversion between entities and DTOs.
Repository	Spring Data JPA interfaces with ready-made CRUD methods and derived queries.
Entity	Java classes mapped to database tables with constraints and relations.
DTO	Records that define exactly what the API accepts (request) and returns (response).
Exception	Custom exceptions and one handler that returns clear JSON errors.

5. Project Structure

The repository has two folders, backend and frontend. The backend is a Maven project with the base package com.sipmp.backend.

```
SIPMP/
  backend/
    pom.xml, mvnw, mvnw.cmd
    src/main/java/com/sipmp/backend/
      SipmpBackendApplication.java
      config/       SecurityConfig
      controller/   Auth, Student, Company, Opportunity, Application,
                    Interview, Dashboard, Reports controllers
      service/      Student, Company, Opportunity, Application,
                    Interview, Dashboard services
      repository/   Student, Company, Opportunity,
                    StudentApplication, Interview repositories
      entity/       Student, Company, Opportunity, StudentApplication,
                    Interview, AuditedEntity and four enums
      dto/          13 request and response records
      exception/    ApiExceptionHandler, ConflictException,
                    ResourceNotFoundException
    src/main/resources/ application.properties, application-test.properties
    src/test/java/com/sipmp/backend/ SipmpApiIntegrationTest
  frontend/         Next.js application
  README.md
```

Package	Classes	Contents
config	1	SecurityConfig
controller	8	AuthController, StudentController, CompanyController, OpportunityController, ApplicationController, InterviewController, DashboardController, ReportsController
service	6	StudentService, CompanyService, OpportunityService, ApplicationService, InterviewService, DashboardService
repository	5	StudentRepository, CompanyRepository, OpportunityRepository, StudentApplicationRepository, InterviewRepository
entity	10	Student, Company, Opportunity, StudentApplication, Interview, AuditedEntity, ApplicationStatus, InterviewStatus, OpportunityStatus, OpportunityType
dto	13	Request and response records for each module, DashboardResponse and CurrentUserResponse
exception	3	ApiExceptionHandler, ConflictException, ResourceNotFoundException

6. Configuration

6.1 Maven dependencies

The pom.xml uses spring-boot-starter-parent 3.5.7 and Java 17. The dependencies are spring-boot-starter-web, spring-boot-starter-data-jpa, spring-boot-starter-validation, spring-boot-starter-security, mysql-connector-j (runtime), lombok (optional), and spring-boot-starter-test and h2 (test scope).

6.2 Application properties

Settings are stored in src/main/resources/application.properties. Values that differ between machines are read from environment variables, so no password has to be written in the source code. The important settings are shown below; the database URL, user name and password are supplied for the local or cloud MySQL server.

```
spring.application.name=sipmp-backend
server.port=${SERVER_PORT:8080}

spring.datasource.url=<JDBC URL of the MySQL database>
```

```
spring.datasource.username=<database user>
spring.datasource.password=<database password>

spring.jpa.hibernate.ddl-auto=${JPA_DDL_AUTO:update}
spring.jpa.open-in-view=false
spring.jackson.serialization.write-dates-as-timestamps=false

sipmp.cors.allowed-origin=${FRONTEND_ORIGIN:http://localhost:3000}
sipmp.security.staff-username=${SIPMP_STAFF_USERNAME:staff1}
sipmp.security.staff-password=${SIPMP_STAFF_PASSWORD:...}
sipmp.security.student-accounts=${SIPMP_STUDENT_ACCOUNTS:}
sipmp.security.recruiter-accounts=${SIPMP_RECRUITER_ACCOUNTS:}
```

Setting	Meaning
ddl-auto = update	Hibernate creates or updates the tables from the entity classes at startup. No sample data is inserted.
open-in-view = false	Database sessions are limited to the service layer, which avoids accidental lazy loading in controllers.
write-dates-as-timestamps = false	Dates and times are written in readable ISO format, for example 2026-11-30.
sipmp.cors.allowed-origin	The only web address (the Next.js frontend, port 3000) allowed to call the API from a browser.
sipmp.security.*	Staff, student and recruiter login accounts, read from environment variables at startup.

Security note. Database passwords and account passwords must be supplied through environment variables or a private configuration file that is not committed to GitHub. If a real password was ever written in application.properties, it should be changed on the database server and replaced by a variable such as ${DB_PASSWORD}.

6.3 Test configuration

The file application-test.properties switches the application to an in-memory H2 database in MySQL mode with ddl-auto = create-drop. The integration test activates this profile, so tests never touch the real MySQL data.

6.4 Environment variables and start-up

Variable	Use
DB_URL, DB_USERNAME, DB_PASSWORD	Connection to the MySQL database (as given in the README)
FRONTEND_ORIGIN	Allowed frontend address for CORS

Variable	Use
SIPMP_STAFF_USERNAME / PASSWORD	Staff account (Placement Officer and administration tasks)
SIPMP_STUDENT_ACCOUNTS	Student accounts as username:password pairs separated by semicolons; the username must be the e-mail of the student profile
SIPMP_RECRUITER_ACCOUNTS	Recruiter accounts; the username must be the contact e-mail of the company

```
Set-Location .\backend
.\mvnw.cmd spring-boot:run
```

The backend starts on http://localhost:8080. Hibernate creates the five tables at the first start.

7. Entities and Database Mapping

The entity classes are the Java form of the database tables. The implemented database is a focused version of the design of Industrial Assignments 5 and 6: it keeps the five core tables needed for the placement workflow. User identity is handled by Spring Security instead of a users table, and the history, resume, offer and preference tables are listed as future enhancements in Section 11.

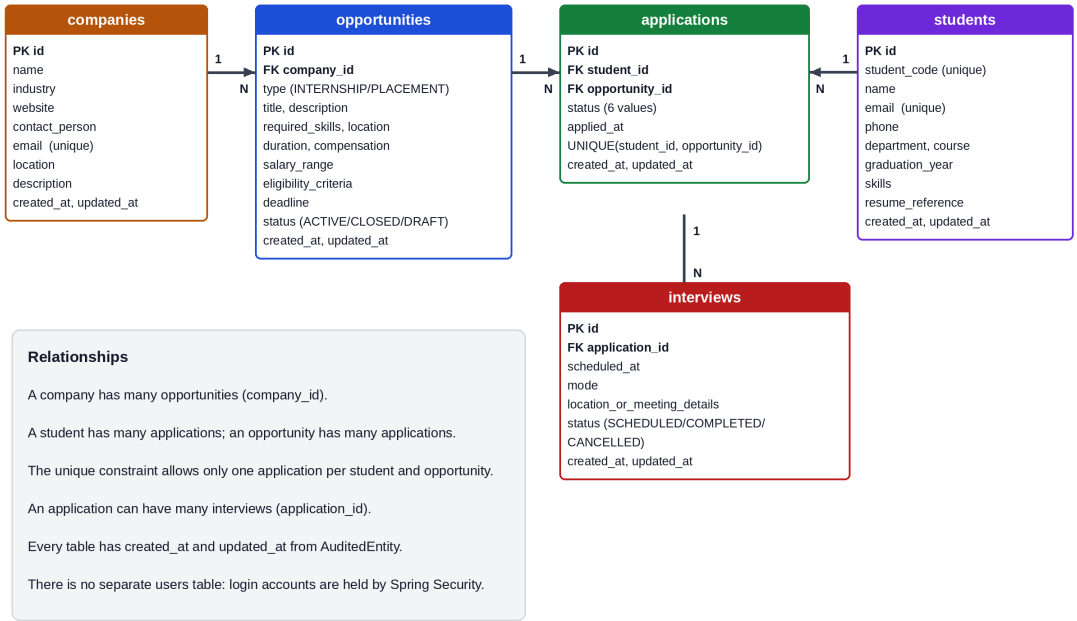


Figure 2: Entity-relationship diagram of the implemented database

Table (entity)	Main columns	Constraints and indexes

Table (entity)	Main columns	Constraints and indexes
students (Student)	id, student_code, name, email, phone, department, course, graduation_year, skills, resume_reference	student_code unique; email unique (uk_student_email); index on department
companies (Company)	id, name, industry, website, contact_person, email, location, description	email unique (uk_company_email); index on industry
opportunities (Opportunity)	id, type, title, description, required_skills, location, duration, compensation, salary_range, eligibility_criteria, deadline, status, company_id	FK to companies; index on (type, status) and on deadline
applications (StudentApplication)	id, student_id, opportunity_id, status, applied_at	FKs to students and opportunities; unique (student_id, opportunity_id); index on status
interviews (Interview)	id, application_id, scheduled_at, mode, location_or_meeting_details, status	FK to applications; index on scheduled_at

7.1 Enumerations

Enum	Values
OpportunityType	INTERNSHIP, PLACEMENT
OpportunityStatus	ACTIVE, CLOSED, DRAFT
ApplicationStatus	APPLIED, UNDER_REVIEW, SHORTLISTED, INTERVIEW_SCHEDULED, SELECTED, REJECTED
InterviewStatus	SCHEDULED, COMPLETED, CANCELLED

Enums are stored as text (@Enumerated(EnumType.STRING)), so the values stay readable in the database and do not change if the order of the enum is changed.

7.2 Timestamps with AuditedEntity

All entities extend one base class that fills createdAt when a record is inserted and updates updatedAt on every change. This gives the created and updated timestamps required for every table in Industrial Assignment 5 without repeating code.

```
@Getter
@MappedSuperclass
public abstract class AuditedEntity {
```

```
    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @Column(nullable = false)
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = createdAt;
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

7.3 Example: Student entity

The entity combines JPA mapping (table name, index, unique constraint, column length) with validation annotations. @Getter, @Setter and @NoArgsConstructor from Lombok remove the need to write getters and setters.

```
@Entity
@Table(name = "students", indexes = @Index(name = "idx_student_department",
columnList = "department"),
        uniqueConstraints = @UniqueConstraint(name = "uk_student_email", columnNames
= "email"))
@Getter
@Setter
@NoArgsConstructor
public class Student extends AuditedEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Size(max = 30)
    @Column(nullable = false, unique = true, length = 30)
    private String studentCode;

    // ... name, email, phone, department, course, graduationYear, skills, resumeReference
}
```

7.4 Relations: Opportunity and StudentApplication

Relations use @ManyToOne with lazy loading and named foreign keys. An opportunity belongs to one company; an application links one student with one opportunity. The unique constraint uk_student_opportunity is the database-level protection against duplicate applications (business rule BR-06).

```
@Entity
@Table(name = "applications",
      indexes = @Index(name = "idx_application_status", columnList = "status"),
      uniqueConstraints = @UniqueConstraint(name = "uk_student_opportunity",
columnNames = {"student_id", "opportunity_id"}))
@Getter
@Setter
@NoArgsConstructor
public class StudentApplication extends AuditedEntity {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @NotNull
   @ManyToOne(fetch = FetchType.LAZY, optional = false)
   @JoinColumn(name = "student_id", nullable = false, foreignKey = @ForeignKey(name =
"fk_application_student"))
   private Student student;

   @NotNull
   @ManyToOne(fetch = FetchType.LAZY, optional = false)
   @JoinColumn(name = "opportunity_id", nullable = false, foreignKey =
@ForeignKey(name = "fk_application_opportunity"))
   private Opportunity opportunity;

   @NotNull
   @Enumerated(EnumType.STRING)
   @Column(nullable = false, length = 30)
   private ApplicationStatus status = ApplicationStatus.APPLIED;

   @NotNull
   @Column(nullable = false)
   private LocalDateTime appliedAt = LocalDateTime.now();
}
```

8. Repositories

Each repository extends JpaRepository, which provides save, findById, findAll, delete and count. Extra methods are written as derived queries: Spring creates the SQL from the method name. One custom JPQL query is used for the placement statistics.

Repository	Extra methods
StudentRepository	findByEmailIgnoreCase, existsByStudentCode, existsByEmail, existsByStudentCodeAndIdNot, existsByEmailAndIdNot
CompanyRepository	existsByEmail, existsByEmailAndIdNot
OpportunityRepository	findAllByOrderByCreatedAtDesc, findByTypeOrderByCreatedAtDesc, countByTypeAndStatus
StudentApplicationRepository	existsByStudentIdAndOpportunityId, findAllByOrderByAppliedAtDesc, findAllByStudent_EmailOrderByAppliedAtDesc, findAllByOpportunity_Company_EmailOrderByAppliedAtDesc, countByStatus, countByOpportunityTypeAndStatus (JPQL)
InterviewRepository	findAllByOrderByScheduledAtDesc

```
public interface StudentRepository extends JpaRepository<Student, Long> {
    java.util.Optional<Student> findByEmailIgnoreCase(String email);
    boolean existsByStudentCode(String studentCode);
    boolean existsByEmail(String email);
    boolean existsByStudentCodeAndIdNot(String studentCode, Long id);
    boolean existsByEmailAndIdNot(String email, Long id);
}
```

```
public interface StudentApplicationRepository extends JpaRepository<StudentApplication,
Long> {
    boolean existsByStudentIdAndOpportunityId(Long studentId, Long opportunityId);
    List<StudentApplication> findAllByOrderByAppliedAtDesc();
    List<StudentApplication> findAllByStudent_EmailOrderByAppliedAtDesc(String email);
    List<StudentApplication>
findAllByOpportunity_Company_EmailOrderByAppliedAtDesc(String email);
    long countByStatus(ApplicationStatus status);

    @Query("""
            select count(application) from StudentApplication application
            where application.opportunity.type = :type and application.status = :status
            """)
    long countByOpportunityTypeAndStatus(
```

```
        @Param("type") OpportunityType type, @Param("status") ApplicationStatus
status);
}
```

The "IdNot" methods are used when a record is updated, so that a student or company can keep its own e-mail but cannot take the e-mail of another record. The methods that walk through properties (findAllByStudent_Email... and findAllByOpportunity_Company_Email...) let a student see only own applications and a recruiter see only the applications of the own company.

9. DTOs and Validation

Entities are never sent directly to the client. Request DTOs are Java records that carry the input and the validation rules; response DTOs return only the fields that the client needs. This keeps the API stable and prevents users from setting fields such as id or createdAt.

DTO	Key validation rules
StudentRequest	studentCode, name, department, course required with maximum lengths; email required and valid; graduationYear required
CompanyRequest	name, industry, contactPerson, location required; email required and valid; maximum lengths on text fields
OpportunityRequest	type, title, description, location, deadline, status and companyId required; compensation 0 or more
ApplicationRequest	studentId and opportunityId required
ApplicationStatusRequest	status required (one of the six ApplicationStatus values)
InterviewRequest	applicationId, scheduledAt, mode (maximum 30 characters) and status required
Response records	StudentResponse, CompanyResponse, OpportunityResponse, ApplicationResponse, InterviewResponse, DashboardResponse, CurrentUserResponse

```
public record StudentRequest(
      @NotBlank @Size(max = 30) String studentCode,
      @NotBlank @Size(max = 120) String name,
      @NotBlank @Email @Size(max = 160) String email,
      @Size(max = 30) String phone,
      @NotBlank @Size(max = 100) String department,
      @NotBlank @Size(max = 120) String course,
      @NotNull Integer graduationYear,
      @Size(max = 2000) String skills,
```

```
        @Size(max = 500) String resumeReference) {
}
```

10. Security Configuration

The SecurityConfig class secures every URL under /api. The portal uses HTTP Basic authentication: the client sends the user name and password in the Authorization header with each request, and the server checks them against the in-memory accounts. Passwords are encoded with BCrypt when the application starts and are never stored in plain text. There are three roles.

Role	Account name rule	Main permissions
STAFF	Configured staff account	Manage students, companies, opportunities, applications, interviews; view dashboard and reports
STUDENT	E-mail of the student profile	View own profile, browse active opportunities and companies, apply, view own applications
RECRUITER	Contact e-mail of the company	Manage opportunities of the own company, review its applications, update status, manage its interviews

```
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(csrf -> csrf.disable())
                .cors(Customizer.withDefaults())
                .authorizeHttpRequests(authorize -> authorize
                        .requestMatchers("/api/auth/me").authenticated()
                        .requestMatchers(HttpMethod.GET,
"/api/students/me").hasRole("STUDENT")
                        .requestMatchers("/api/opportunities/**",
"/api/companies/**").hasAnyRole("STUDENT", "RECRUITER", "STAFF")
                        .requestMatchers("/api/applications/**").hasAnyRole("STUDENT",
"RECRUITER", "STAFF")
                        .requestMatchers("/api/interviews/**").hasAnyRole("RECRUITER", "STAFF")
                        .requestMatchers("/api/**").hasRole("STAFF")
                        .anyRequest().permitAll())
                .httpBasic(Customizer.withDefaults())
                .build();
    }
```

URL rules are written from the most specific to the most general. Method-level rules (@PreAuthorize) in the controllers then restrict individual actions, for example only STAFF may delete a company. If a role breaks a rule, Spring Security returns 401 (not logged in or wrong password) or 403 (logged in but not allowed).

```
   CorsConfigurationSource corsConfigurationSource(
         @Value("${sipmp.cors.allowed-origin}") String allowedOrigin) {
      CorsConfiguration configuration = new CorsConfiguration();
      configuration.setAllowedOrigins(List.of(allowedOrigin));
      configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
      configuration.setAllowedHeaders(List.of("Authorization", "Content-Type"));
      configuration.setAllowCredentials(true);
      UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
      source.registerCorsConfiguration("/api/**", configuration);
      return source;
   }
```

CSRF protection is disabled because the API is stateless and uses an Authorization header instead of cookies. CORS allows only the configured frontend origin.

11. Comparison with the Earlier Design

The earlier assignments describe the complete portal (16 tables and 28 tasks). The backend developed for this project implements the working core of that design. The table shows how the two relate so that the report stays consistent.

Industrial Assignment 6 design	Implemented in the backend	Remark
users, login_sessions, user_access_history	Spring Security in-memory accounts with roles	Login by HTTP Basic; session and access-history tables are a future enhancement
student_profiles	students	Contains department, course, graduation year, skills and resume reference
student_resumes, student_edit_history	resumeReference field	File upload and edit history are future enhancements
companies	companies	Implemented
opportunities	opportunities	Implemented with type, status, deadline and eligibility text

Industrial Assignment 6 design	Implemented in the backend	Remark
opportunity_edit_history, opportunity_deletion_history	Not implemented	Closing is done by status CLOSED; delete removes the record
applications, application_status_history	applications	Implemented; status history is a future enhancement
interviews	interviews	Implemented
offers	Status SELECTED	A separate offer record is a future enhancement
dashboard_preferences, report_history	Live dashboard and reports API	Figures are calculated from live data

12. Conclusion

The backend foundation of the portal was built with Spring Boot 3, Java 17 and MySQL. It uses a clear layered structure, five entities with keys, relations, unique constraints and indexes, DTO records with validation, and Spring Security with BCrypt and three roles. These parts are used by the services, controllers and exception handling described in Industrial Assignment 11.